

Practica de Laboratorio 4 de Jesús Rachadell 31421494

1. Diagrama de Interrupción

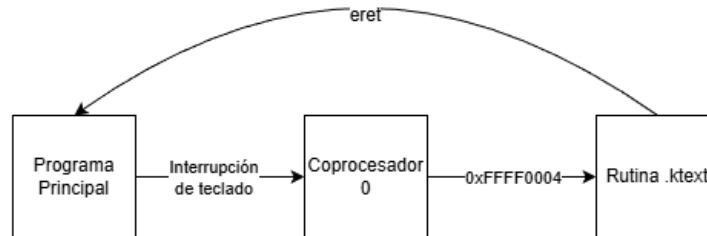


Figura 1: Diagrama de interrupciones

El sistema E/S por interrupciones permite un procesamiento asíncrono. En estos sistema el hardware permanece pasivo, y al detectarse una pulsación del teclado, el procesador detiene lo que está haciendo guarda la dirección de retorno en el EPC, y ejecuta lo que está en .ktext, que es donde se maneja la interrupción, y al final regresa con *eret* para que el procesador siga con lo que estaba haciendo.

2. Sondeo vs Interrupciones

Característica	Gestión por Sondeo	Gestión por Interrupciones
Iniciativa	El procesador pregunta al periférico constantemente por datos	El periférico avisa al procesador que ya tiene un evento listo
Uso de CPU	Es ineficiente, el procesador se queda esperando y desperdicia ciclos	Es eficiente, el procesador realiza otras tareas y solo atiende cuando es necesario
Sincronización	Sincrónica, el programa debe pasar por la instrucción para leer el dato	Asincrónica, el evento puede pasar en cualquier momento
Complejidad	Es más fácil de diseñar	Requiere manejar CoProc0

Cuadro 1: Tabla de diferencias

3. Ventajas de las Interrupciones

El modelo de gestión por interrupciones soluciona el problema de la espera activa, permitiendo al procesador ejecutar otras tareas mientras espera al periférico que mande un evento que luego es procesador por el handler, esto le permite ser más eficiente a su vez de trabajar la multitarea.

4. Registros especiales de Coproc0

Se usan principalmente tres registros del Coproc0: Status(\$12), Cause(\$13) y EPC(\$14)

4.1. Registro Status

Se maneja al principio de la ejecución como \$12, lo primero para habilitar las interrupciones de cualquier tipo es activar el bit 0 (IE-Interrupt Enabled) que funciona como un switch maestro. Los bits 8-15 funcionan como filtros individuales para cada periférico, para el teclado por ejemplo, el bit que lo maneja es el 11,

4.2. Registro Cause

Se usa para determinar si la interrupción fue por parte de un hardware interno o no, para ello se utilizan los bits 2-5, si son 0 significa que la interrupción fue 100 % de un hardware externo.

4.3. Registro EPC

El procesador no sabe cuando una interrupción va a ocurrir es debido a eso que para que el procesador no pierda el hilo de lo que estaba haciendo se guarda el PC en el registro EPC, una vez *eret* el PC se restaura y el CPU sigue con lo que estaba haciendo previamente.

5. Guardar el contexto

Al entrar a una rutina .ktext los valores pueden modificarse y al volver corromper la tarea que se estaba realizando. Sí no se usa otro registro que no sean los \$k0 o \$k1 el único registro que es importante guardar es el \$at ya que usa en todas las pseudoinstrucciones como un registro pasajero, al volver a la tarea que estaba haciendo el CPU necesita el valor de \$at que estaba usando para no ocasionar ningún error.

6. Excepciones en MIPS32

Desbordamiento Aritmético: Sucede cuando una operación con signo (add, sub, etc) genera un resultado demasiado grande para ser representado en 32bits

Fallo de Dirección: Sucede cuando se intenta acceder a una dirección de memoria no valida

Instrucción invalida: Sucede cuando el procesador intenta ejecutar un opcode que no forma parte del conjunto de instrucciones, comúnmente ocasionado por intentar leer datos como direcciones.

Syscall: Intencionalmente utilizada para solicitar un servicio al procesador.

En IF puede haber una excepción del tipo Fallo de dirección, esto sucede cuando el PC no apunta a una dirección valida. En ID puede una excepción del tipo Instrucción invalida, cuando el opcode no es reconocible. En Ex pueden ocurrir un Desbordamiento aritmético como se explicó antes. En MEM, pueden a su vez haber una excepción del tipo Fallo de dirección si una instrucción de lectura o escritura intenta leer una dirección de memoria invalida.

7. Excepciones e Interrupciones

Una de las principales diferencias es que las excepciones son eventos internos y las interrupciones son eventos externos, eso significa que a su vez las excepciones son síncronas ya que son predecibles en el pipeline, en cambio las interrupciones pueden ocurrir en cualquier momento y son asíncronas como se discutió antes.

1. **Vector único:** Todas las excepciones e interrupciones van a la misma dirección (en el caso de los ejercicios la 0x80000180) y se lee el registro Cause para determinar que la ocasiono
2. **Vector de interrupciones:** Cada excepción como interrupción posee un espacio reservado. Esto hace que sea más rápido al no tener que determinar la causa.

Al momento de la interrupción, suceden tres cosas, el pipeline se congela, los estados se guardan y se carga en PC la dirección 0x80000180 (en caso de los ejercicios). Aquí es importante el registro EPC porque como se mencionó antes este guarda el PC anterior a la interrupción, una vez terminado lo restaura.

8. Habilitación de interrupciones en dispositivos y procesador

Para habilitar el teclado se debe activar el bit 1 del valor que se encuentra en 0xFFFF0000, esto hace que empiece a registrar el teclado. Para habilitar la pantalla se debe activar el bit 1 del valor que se encuentra en 0xFFFF0008. Por ultimo para el registro Status se debe activar el bit 0 que activará la inclusión de todas las interrupciones, y dependiendo de que se vaya usar se debe activar uno de los bits 8-15, en caso del teclado es el bit 11.

Sí se habilitan los periféricos pero no el procesador (bit 0 de Status), el programa leerá los datos del periférico pero el procesador nunca detendrá el programa dejando esa entrada inutilizada.

9. Procesamiento de Interrupciones

Al haber una interrupción de reloj lo primero que hace el CPU es congelar el pipeline, luego inhabilita el bit 0 de Status para que no haya más interrupciones mientras se ejecuta el servicio, copia la dirección de PC en EPC, y carga en PC la dirección del vector de interrupciones 0x80000180. A partir de ahí primero se guarda el \$at en caso de que el procesador estaba usando una pseudoinstrucción, luego identifica que clase de interrupción fue usando el registro Cause, se ejecuta el evento y se finaliza restaurando el contexto (incluyendo el bit 0 de Status), en el caso de los ejercicios el \$at, y carga en PC el registro EPC para volver donde estaba, reanudando la tarea.

Dependiendo de que se haga en la rutina de servicio se debe guardar los registros que se usaron dentro ya que estos podrían estar siendo utilizados afuera de la rutina y no pueden ser comprometidos, en el caso de \$at es obligatorio por el uso de pseudoinstrucciones. Como se explicó antes EPC guarda la dirección del PC antes de entrar a la rutina esto hace que el CPU pueda volver a donde estaba una vez terminada la rutina.

10. Interrupciones de reloj y control de ejecución

Utilizando la interrupción por reloj podemos hacer que un programa tenga un tiempo límite para ejecutarse, solo debemos elegir un tiempo máximo y hacer un bucle donde se compruebe primero el tiempo de ejecución y luego la condición de salida, así si no se consigue la condición de salida el programa saldrá de todas formas, permitiendo al programa terminar sin problema.

Una vez ejecutado el `li $v0, 10, syscall` que termina con el programa, el software del Kernel cancela el temporizador asociado a ese programa, dejándolo listo para el siguiente programa en cola.

11. Análisis y Discusión de los resultados

Procedo a mostrar la ejecución de los ejercicios 1 y 3 respectivamente y como se comportan en distintas situaciones:

11.1. Consideraciones

Primero se mostrará el funcionamiento del ejercicio 1 e inmediatamente después el funcionamiento del ejercicio 3.

11.2. Preparación

Antes de entrar en el bucle principal los dos ejercicios necesitan habilitar el teclado y las interrupciones como tal

```
main:
    # Habilita la entrada de teclado
    li $t0, 0xFFFF0000
    lw $t1, 0($t0)
    ori $t1, $t1, 0x02
    sw $t1, 0($t0)
    # Configuración del coproc0
    mfc0 $a0, $12
    ori $a0, $a0, 0x801
    mtc0 $a0, $12
```

Figura 2: Inicializando teclado y Status

Luego para poder escribir en el teclado usaremos la siguiente herramienta de Mars:

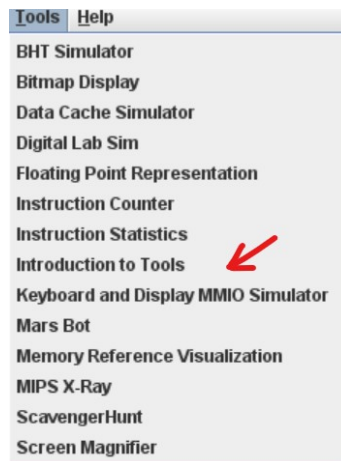


Figura 3: Keyboard

11.3. Ejercicio 1: buffer circular

Cubriré los principales casos de este ejercicio, su funcionamiento debe guardar letras de la A-Z en un buffer circular, es decir una vez llegado al límite (100 para mi caso) sobrescribe las primeras letras guardadas. Para ello opté por un ciclo que espera 20 segundos antes de imprimir, al hacerlo borrar todo registro del buffer para ser usado en una nueva iteración. En la siguientes imágenes se puede ver su funcionamiento en las distintos casos:

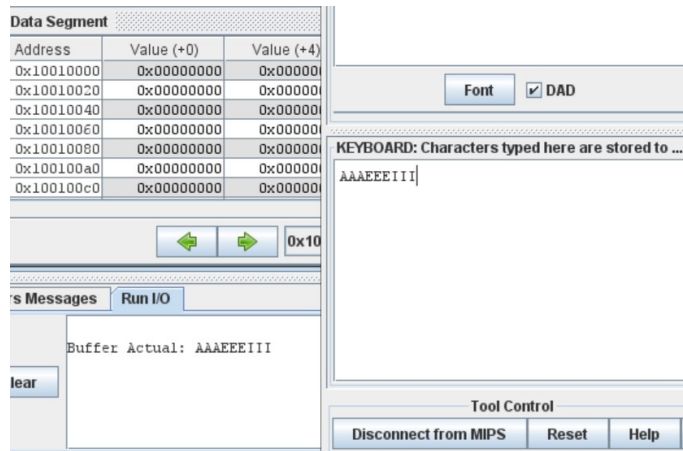


Figura 4: No sobrexcede el límite

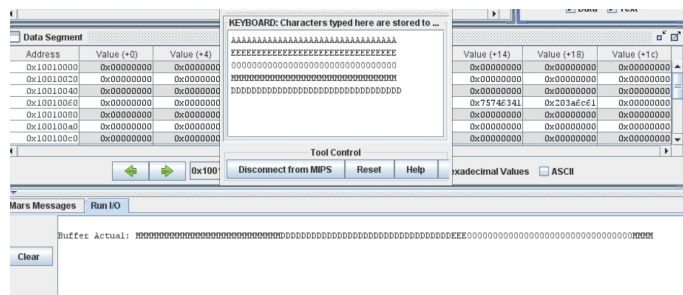


Figura 5: Sobrescribe al excederse el límite

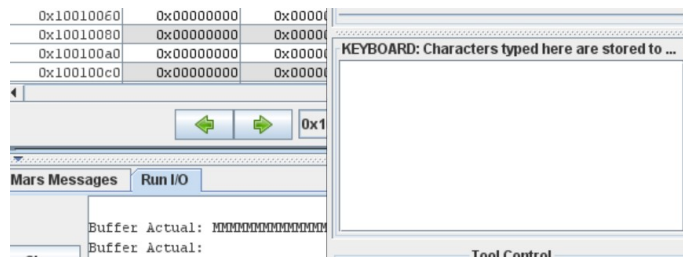


Figura 6: Al no haber nada en el buffer no imprime nada

11.4. Ejercicio 3: Semáforo

Para este ejercicio lo más importante es llevar un control de los ciclos de reloj y de como se manejan las interrupciones, para ello se debe tener un control de estados para saber que ciclo debe ejecutarse, la interrupción solo será validad en el estado repaso (0) así solo una vez activado 's' pueda empezar la secuencia como se muestra en continuación:

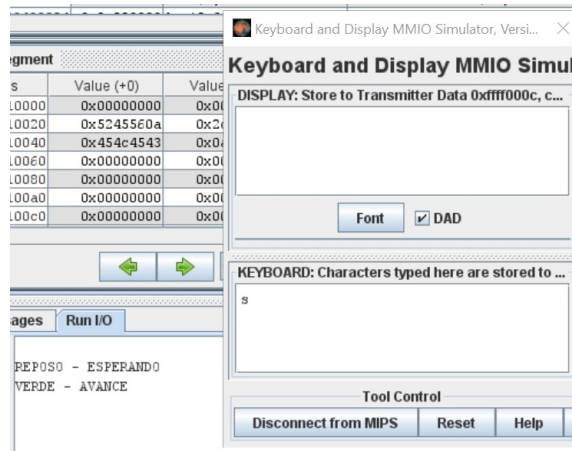


Figura 7: Presionando 's' para empezar

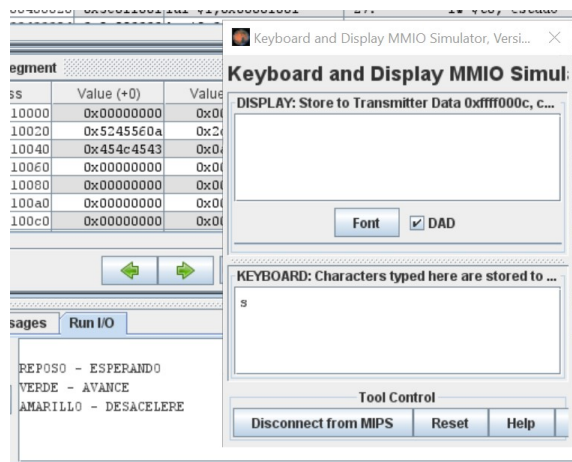


Figura 8: Pasa por Verde 20 segundos

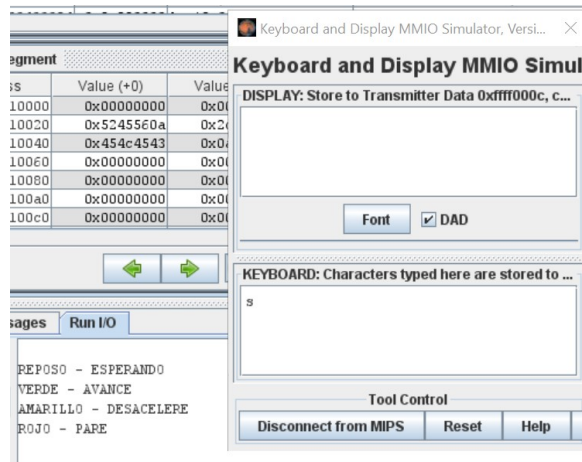


Figura 9: Pasa por Amarillo 10 segundos

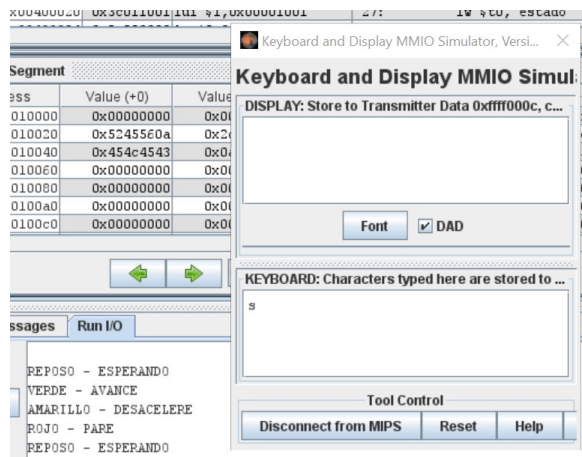


Figura 10: Pasa a Rojo 30 segundos

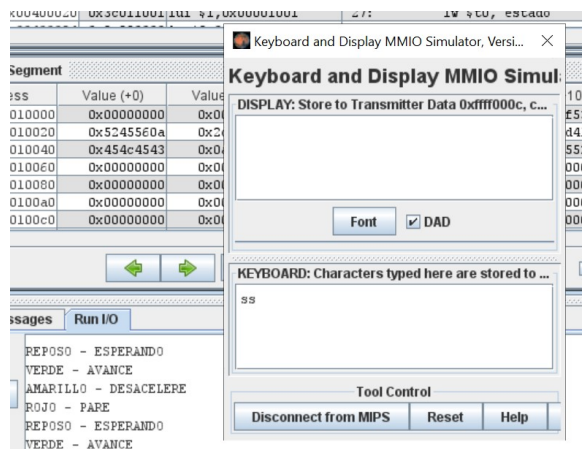


Figura 11: Vuelve al estado inicial listo para volver a empezar la secuencia